Tabletop Manipulation With A Physical Robot Arm Using Vision-Language-Action Models

Linda Zeghib, Khadija Gamaleldin, Gautham Vishnu Chandran and Rohan Mahale

***Abstract*— Recent advancements in Vision-Language-Action (VLA) models have demonstrated significant potential for generalizing robotic manipulation using natural language. However, deploying these models in high-frequency, closed-loop control systems poses practical challenges regarding inference latency and domain adaptation. In this project, we develop and evaluate a manipulation pipeline for a Franka Emika Panda robot operating in a ROS 2 environment. Our system maps external camera observations and text instructions directly to continuous robot actions. To assess the adaptation, we evaluated the system on a block-in-box manipulation task across various training intervals. While initial models successfully learned directional orientation, complete physical execution was initially hindered by the replication of human teleoperation pauses, causing the arm to freeze before grasping. By algorithmically filtering these near-zero action samples, we successfully overcame these teleoperation biases. The final fine-tuned model achieved autonomous completion of the manipulation task, requiring only a minor manual override to bypass residual hesitation, thereby validating the practical viability of deploying generalized VLA models onto physical control systems.**

I. INTRODUCTION

Classical robotics has been largely characterized by highly rigid, modular pipelines where visual perception, task planning, and motion control operate as isolated systems. Although they remain very efficient for structured industrial environments, they usually struggle to adapt to unstructured tasks or natural human interaction. Integrating these robots into robust and generalizable agents has continued to be a challenge until recent years. Vision-Language-Action (VLA) models have since emerged as a shift in the robotics industry, offering frameworks that translate visual observations and language instructions into physical actions.

Despite these major advancements, there is still a significant gap between large-scale models and more practical hardware deployments. Individual robot setups have different camera properties, lighting conditions, and control frequencies, which foundational VLAs lack the context of. The motivation of this project is to bridge the gap between generalized models and hardware deployment. By fine-tuning an existing VLA model on a custom, hardware-specific dataset, our work investigates the practical engineering challenges of translating a generalized VLA model into a localized control system.

The overarching objective of this project is to develop, integrate, and evaluate a fully functional, language-conditioned manipulation stack for a Franka Emika Panda robotic arm. Rather than developing a new model from scratch, the project focuses on the systems engineering required to adapt an existing model for continuous, real-world task execution. The specific goals include:

1) **Establish a synchronized data pipeline:** The first goal for this project is to successfully design and implement a ROS 2-based data collection architecture to record synchronized episodes for our dataset. It should capture natural language instructions, joint states, and RGB visual observations.
2) **Implement hardware-specific fine-tuning:** The second goal is to adapt a generalized VLA model (such as OpenVLA) to our setup's specific physical restraints, ranging from camera properties and control frequencies to robot-specific hardware.
3) **Integrate the model:** The next goal is to implement our final fine-tuned VLA model onto the physical robot with real-time visual and semantic inputs and translate the action tokens into continuous commands.
4) **Evaluate the adaptation:** Our final objective is to assess the practicality of our system and identify the challenges of the deployed pipeline.

A. Outline of the Paper

To address the previously mentioned objectives, the remainder of this paper is structured into five sections. The next section covers the fundamental theoretical background information, exploring the core architecture of Transformers, Vision-Language and VLA models, and the ROS 2 middleware necessary for deployment. Section III details the methodology, introducing the specific hardware choices and components, as well as covering the overall data collection design. Section IV dives into the practical implementation, describing the data processing pipeline in more detail and covering the model integration and fine-tuning procedures. Section V presents the final results and discussion by evaluating the system's performance and the challenges faced during execution. Finally, the last chapter concludes the paper and proposes further research for future work.

II. FUNDAMENTALS

This section outlines the main architecture behind VLA models and provides the background information necessary for complete understanding of the paper.

A. Transformers & Attention

The Transformer architecture was introduced by the influential paper titled "Attention is all you need" [1]. It is a type of neural network architecture designed to understand context and meaning by tracking relationships in sequential data. Transformers completely revolutionized the

Project Course Control Engineering, Summer Term 2026. Supervisors: Sebastian Hirt, Konstantin Preußer, Hendrick Alsmeier

field of machine learning, since previously, learning-based methods processed sequences like text sequentially, word by word, using architectures like Recurrent Neural Networks (RNNs). This architecture faced several limitations, including limited parallelization due to sequential processing [1] and vanishing or exploding gradient problems, which hindered its ability to effectively model long-range dependencies [2]. Transformers process everything at once in parallel through a defining feature called the Self-Attention mechanism. It is the mathematical engine that allows neural networks to understand context. A word in a sentence rarely means much in isolation and without context for the model. Its true meaning depends entirely on its surroundings. Figure 1 illustrates and summarizes the transformer architecture.

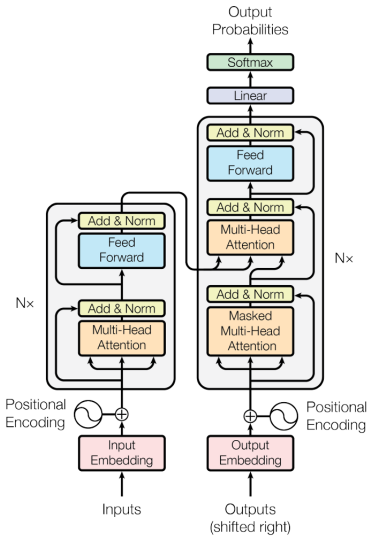


Fig. 1: This diagram illustrates the Transformer model architecture [1]. The encoder block, located on the left side, processes the initial input sequence to map its context. The decoder block on the right takes this contextual mapping and uses it to generate the final output sequence.

An important part of the transformer is the shared workload between its encoder and decoder. They are responsible for comprehending an input and generating an output. At a high level, the encoder's purpose is to process the entire input sequence simultaneously to build a contextual map. The decoder's job is to take the encoder's compressed understanding and translate it into the final output sequence. Specifically, the encoder includes N layers, with each layer containing two sub-layers: a multi-head self-attention mechanism and a fully connected feed-forward network [1]. The decoder features a similar structure but handles sequential dependencies differently. During inference, the decoder operates autoregressively, meaning it generates token 1, then uses token 1 to generate token 2, and so on. The decoder must not refer to future information, which is enforced through mathematical masking. During auto-regressive generation, this mask ensures the decoder only pays attention to tokens it has already generated, whereas during training, the mask ensures the model only attends to the preceding true labels from the training data rather than peeking ahead at future target tokens.

B. Vision Transformers & Feature Extraction

A vision transformer operates by applying the standard transformer architecture, which was originally designed for natural language, directly to images with minimal modifications. Instead of using traditional convolutional networks, the model splits an image into a grid of fixed-size patches, such as 16x16 pixels [3], flattens them, and projects them into linear 1D embeddings. By adding learnable positional embeddings to retain the spatial layout, the model treats these image patches similarly to words in a sentence, feeding the entire sequence into a standard transformer encoder to achieve image recognition when pre-trained on large-scale datasets [3]. Figure 2 depicts the ViT and transformer encoder architecture.

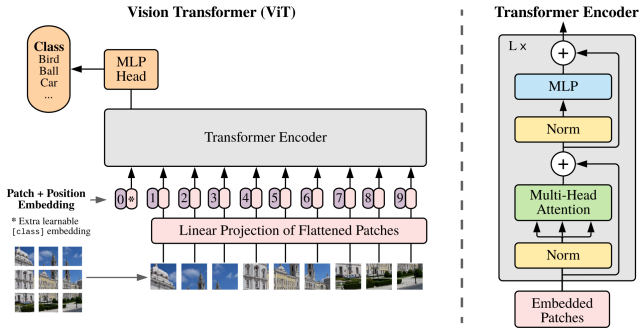


Fig. 2: The Vision Transformer architecture and Transformer Encoder are displayed in this image [3]. The ViT splits an input image into fixed-size patches that are linearly projected and combined with positional embeddings. This sequence is then processed through a standard Transformer Encoder before it outputs the image classification.

One prominent Vision Transformer is DINOv2 (self-DIstillation with NO labels), an updated version of DINO developed by Meta AI [4]. DINOv2 utilizes self-supervised learning, a specialized form of unsupervised learning where the model extracts supervisory signals directly from the structure of the data itself. This approach is highly advantageous because it eliminates the bottleneck of requiring human-annotated external labels. Consequently, the model can be trained on vast amounts of unlabeled image data, allowing it to learn highly robust and generalizable visual features that can match the performance of fully supervised methods [4].

C. Vision-Language Models (VLMs)

At their core, Vision-Language Models (VLMs) are artificial intelligence systems designed to understand, process, and reason across two fundamentally different types of data simultaneously: visual information (images or video) and human language.

Until the development of multimodal architectures in the late 2010s, deep learning models were strictly isolated by modality [5]. Computer vision models had demonstrated high accuracy in specific tasks such as object detection and localization [6], while large language models had recently achieved proficiency in complex natural language generation [7]. However, these unimodal systems could not interact or share cross-domain information. VLMs bridge this modality

gap, allowing a single model to process visual inputs and generate context-aware, natural language responses about the visual data [8].

D. Vision-Language-Action (VLA) Models

VLA models are trained models designed to map robot observations, which is usually done through a camera, and language instructions directly to actions. To fit both natural language responses and robot actions into the same format, VLA models express actions as text tokens, which are then incorporated directly into the training set of the model [9].

The VLA architecture has three encoder streams: visual, linguistic, and state. A transformer-based visual encoder first processes the raw images and creates a sequence of tokens while a language encoder does the same out of natural language instructions. The state encoder tokenizes the kinematic state of the robot such as its joint angles, end-effector states, and gripper status. These embeddings are then fed to a Large Language Model (LLM) that generates a representation of the task, which is further processed by an action decoder to generate the path needed to accomplish the task [10]. A complete visual of this architecture is depicted in Figure 3.

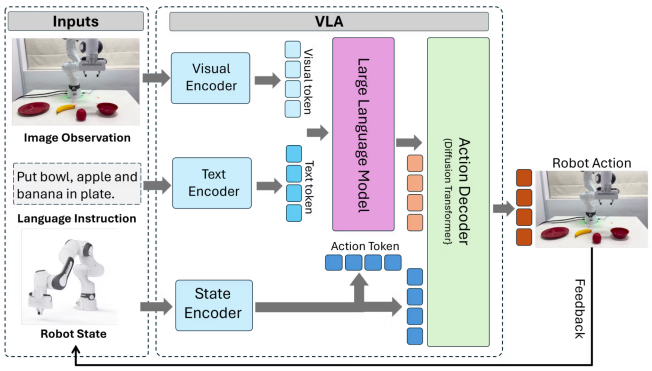


Fig. 3: This diagram illustrates a VLA framework [10] that converts image observations, language instructions, and the robot's kinematic state into discrete tokens using dedicated encoders. These visual and text tokens are processed by an LLM to generate contextual understanding, which an Action Decoder then utilizes alongside state tokens to output specific physical robot actions. Finally, a continuous feedback loop updates the input robot state based on the newly executed actions, enabling dynamic control during manipulation tasks.

VLA-Model Selection

Several key factors guided the selection of the VLA model to maximize the likelihood of successful fine-tuning and reliable inference. Because the project was restricted to publicly available models, three candidates were considered: OpenVLA [11], Octo [12], and π_0/OpenPi [13]. Their source availability, documentation, training-data diversity, hardware requirements, and fine-tuning options are compared in Table IV. Hardware capacity was not a limiting factor for inference, as the available workstation was equipped with an AMD Ryzen 9 5950X processor with 16 cores and 32 threads, 64 GB of system memory, and an NVIDIA GeForce RTX 3090 GPU with 24 GB of VRAM. Consequently, the greater computational requirements of OpenVLA and π_0/OpenPI were considered acceptable. OpenVLA was ultimately selected because it offered the strongest overall software ecosystem, including comprehensive documentation, a PyTorch-based implementation, Hugging Face integration, parameter-efficient fine-tuning support, and extensive community resources.

OpenVLA

OpenVLA's architecture can be understood as a pretrained VLM whose output vocabulary and training procedure have been adapted from language generation to robot-action generation. OpenVLA is built on the Prismatic-7B VLM, which processes a language instruction and an image through two input pathways (a detailed schematic is provided in Appendix Figure 10). In the language pathway, the instruction is processed by the Llama tokenizer [14], which converts the text into discrete token IDs that are subsequently mapped to language-token embeddings.

In the visual pathway, the input image is processed independently by the pretrained DINOv2 and SigLIP vision encoders. Each encoder produces a feature vector for every image patch. The corresponding DINOv2 and SigLIP patch features are concatenated to combine spatial and semantic visual information. These combined features cannot be passed directly to Llama 2 because their dimensionality and representation space do not match Llama 2's token-embedding space. They are therefore passed through an MLP, or multi-layer perceptron, projector. The projector transforms each combined patch feature into a visual-token embedding with the dimensionality expected by Llama 2.

The visual-token embeddings and language-token embeddings are then passed together to the Llama 2 7B transformer. In an ordinary VLM, the transformer generates text tokens, for example to answer a question or describe a scene. In OpenVLA, the generated tokens instead represent a discretized 7D robot action, where each action dimension is normalized to $[-1, +1]$ and discretized into 256 bins. Each of the seven action dimensions is represented by one action token, and these tokens are generated autoregressively, meaning that they are predicted sequentially, with each prediction conditioned on the previously generated tokens. The action tokens are then de-tokenized into approximate continuous robot-control values. [11]

OpenVLA-OFT

OpenVLA-OFT introduces an Optimized Fine-Tuning recipe that starts from the pretrained OpenVLA checkpoint and changes how the model is fine-tuned and how it predicts robot actions. Observe Figure 11 in the Appendix. Note that this figure shows OpenVLA-OFT+, an augmented version of OpenVLA-OFT, which we will further elaborate on in subsection IV-B. First, the discrete action tokens used by OpenVLA are replaced by directly predicted continuous action values, avoiding the loss of precision caused by discretization.

Second, autoregressive decoding is replaced by parallel decoding, allowing all action dimensions to be predicted simultaneously in a single forward pass. OpenVLA-OFT also

uses action chunking, meaning that it predicts a sequence of future actions rather than only one action timestep.

Finally, whereas the original OpenVLA is trained using next-token prediction with a cross-entropy loss, OpenVLA-OFT predicts continuous actions through an MLP action head and is trained using an L1 regression objective. OpenVLA-OFT can also process additional inputs, including multiple camera views, such as third-person and wrist-camera images, as well as the robot's proprioceptive state.

These improvements enable substantially faster inference, with approximately a 26× increase in action-generation throughput, more precise action prediction, and higher task success rates. For these reasons, we selected OpenVLA-OFT as our VLA model [15].

E. Action & Robot Learning

While VLMs and VLAs excel at reasoning over visual and textual inputs, deploying them in physical environments requires extending their capabilities to generate concrete robotic actions. Training models like OpenVLA-OFT to successfully translate this reasoning into physical movements necessitates massive and diverse datasets of real-world robot trajectories. To provide the foundational data required for this action generation, we look to large-scale robotic datasets.

Open X-Embodiment

The Open X-Embodiment (OXE) project [16] is a large collaborative effort that aims to support robot learning across many different robotic platforms. The project brings together data from 22 robot embodiments, collected through a collaboration between 21 institutions. These robots demonstrate 527 skills across a wide range of manipulation tasks. Instead of training a separate model for every robot, task, or environment, the goal of OXE is to study whether one model can learn useful behaviors from data collected on many different robots. This is similar to the way large language and vision models are trained on broad and diverse datasets before being adapted to more specific tasks.

OXE provides an important data foundation for generalist robot policies, including VLA models such as OpenVLA. A VLA model can first be pretrained on large-scale robot data using powerful computational resources, and then later fine-tuned for a specific robot setup with much less data and compute. In this way, OXE helps make it more practical to transfer knowledge from one robot, environment, or task to another.

Reinforcement Learning Dataset

To organize the robot data in a consistent way, OXE uses the Reinforcement Learning Dataset (RLDS) format [17]. RLDS is a standard format for storing sequential robot or reinforcement learning data. In this format, a robot trajectory is stored as an episode, and each episode contains an ordered sequence of steps. Each step represents a timestep and can include observations, actions, rewards, discounts, terminal flags, and metadata (see Figure 12a in the Appendix). An observation may include camera images, robot joint states, gripper state, or a language instruction, while the action represents the command that the robot executed.

Although the RLDS format originates from reinforcement learning, its use in Open X-Embodiment does not imply that the VLA model is necessarily trained with a reinforcement learning objective. In this setting, the dataset mainly serves as a standardized format for storing sequential robot demonstrations. These demonstrations can then be used for supervised imitation learning, where the model is trained to predict the recorded robot action from the corresponding visual observation and language instruction.

The RLDS format is designed to be lossless, uniform, and flexible. It is lossless because it preserves the full order of the episode and its steps, rather than reducing the data to independent observation-action pairs. It is uniform because common fields such as observations, actions, rewards, and episode boundary flags are stored using a shared structure across datasets. It is flexible because it does not force every dataset to use the same structure or number or type of features inside those common fields. This is especially important for robotics, where different robots may use different cameras, control commands, and state representations.

F. Robotic Middleware

Robot Operating System (ROS) 2 is an open-source software platform and middleware used to build robot applications [18]. It runs on top of a host operating system and provides the hardware abstraction, device control, communication, and package management necessary to make different robotic components such as cameras, actuators, and controllers work together.

ROS 2 can be described as a distributed system of nodes. A node is a single executable that performs a specific task. For example, one node might read frames from a depth camera, while another node calculates certain movements for a robotic arm [18]. These nodes need to talk to each other, which they do through three primary communication paradigms.

- Topics (Publish/Subscribe): Used for continuous data streams. The camera node "publishes" images to an /image_raw topic, and the arm controller node "subscribes" to that topic to receive the images.
- Services (Request/Response): Used for quick, synchronous, one-off operations. Node A asks Node B to "compute this trajectory" and waits for the exact answer.
- Actions (Goal/Feedback/Result): Goal-oriented and used for long-running, asynchronous tasks. Node A tells Node B to *"move the arm to coordinate X."* Node B executes the movement, providing continuous feedback on its progress until it reaches the result.

III. METHODOLOGY

A. Overall System Architecture

The overall architecture of this project is divided into two primary pipelines: the offline data collection and training process, and the live closed-loop control system. The training pipeline begins with visual data acquisition and state logging, which are managed by a distributed network of ROS 2 nodes. Specifically, dedicated nodes handle the camera stream, the

robot's kinematic state broadcasting, and the language instruction interface, publishing this data to designated ROS topics. The continuous, synchronized stream of these observations is recorded into rosbags. The exact topics are detailed in Section IV-A. To bridge the gap between ROS 2 and the machine learning framework, these rosbags are subsequently converted into the RLDS format. This structured dataset is then used to fine-tune the VLA model.

During live execution, the final system operates as a multimodal control architecture. The model receives a user-defined natural language instruction alongside real-time RGB image observations. The underlying VLA network processes these inputs to predict a sequence of action tokens. Finally, these discrete tokens are translated into continuous kinematic commands and transmitted to the Franka Emika robot arm to execute the physical manipulation task.

B. Robot Arm

The physical execution of the manipulation tasks is performed using a Franka Emika Robot system, provided by the university department. The hardware setup consists of a 7-axis robotic arm, a main control computer, and the electrical two-finger parallel gripper. A critical component for this project's architecture is the Franka Control Interface (FCI), a research-oriented feature that provides a low-level control interface. In order to use the terminal and ROS 2 to directly command joint positions, velocities, and Cartesian poses, the FCI must first be activated. This access is essential for translating the continuous action tokens generated by the VLA model into physical movements with minimal latency [19].

The Franka Emika Panda robot arm was used alongside RViz for visualization and control. RViz is the standard 3D visualizer for ROS. It does not do any computing or controlling on its own but rather simply subscribes to the ROS topics and renders the data in a 3D spatial environment, essentially depicting the robot's digital twin as shown in Figure 4. Utilizing RViz to monitor the camera topic allowed for continuous observation of the robot and its interaction with the object. Evaluating this visual stream directly informed our iterative adjustments, ultimately leading to the optimal and final positioning of the camera.

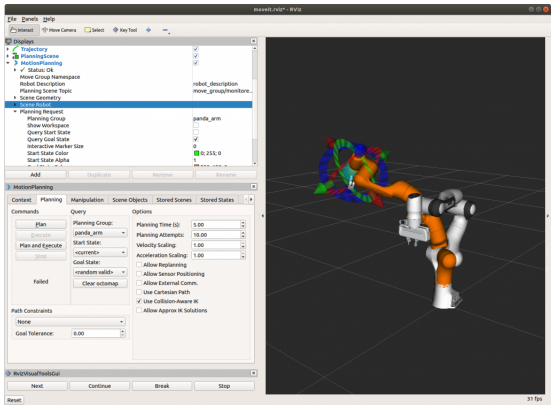


Fig. 4: An example of the Franka Emika Panda robot arm viewed in RViz.

C. Camera Setup

As the camera provides the principal source of visual input for our system, we performed a comprehensive analysis of its specifications, spatial configuration, and system compatibility to ensure the input's quality. The pre-training constraints of OpenVLA necessitated specific camera placement. Since it was trained on the OXE dataset, the majority of the datasets featured fixed, third-person cameras looking directly at the robot's workspace [11].

Given the broad compatibility of standard USB interfaces with the ROS 2 ecosystem, the primary selection criteria for the camera focused on three main factors:

- System Compatibility: Compatibility with Ubuntu Linux to ensure seamless integration with existing ROS 2 nodes.
- Manual Configuration: Strict hardware-level control over imaging parameters. Dynamic features such as auto-focus and auto-exposure introduce visual variance between frames, which can degrade the model's tokenization process and action prediction accuracy.

After extensive research and trials of previously owned web cameras, we decided on the SVPRO 5MP 30FPS USB wide-angle camera [20]. In addition to fulfilling the previously mentioned selection criteria, its light weight and configurable rotation made it ideal for our setup.

To ensure a consistent field of view during data acquisition, we required a camera setup that was both highly stable and easily adjustable. We designed and 3D-printed a custom mount that slides along the vertical aluminum extrusion and can be rigidly locked in place, giving the camera free degrees of movement in the X, Y, and Z directions. Figure 5 displays the final camera setup and the 3D model of the mount used to support it.



Fig. 5: The final camera setup is shown on the left, while the 3D model of the mount is on the right.

D. Object Selection

The choice of object for training our model must align with the physical constraints of the hardware and the visual biases of the pre-trained network in order to maximize the fine-tuning success for our OpenVLA-driven setup. Selecting

a high-contrast, matte cuboid object sized between $30\,\mathrm{mm}$ and $50\,\mathrm{mm}$ would minimize geometric complexity and prove to be an effective starting point for the fine-tuning process [21]. Since the Open X-Embodiment pre-training dataset mainly utilizes controlled, high-contrast lab environments, our implementation of a similar object optimizes the DINOv2 vision encoder's edge detection and prevents out-of-distribution errors.

TABLE I: Final object selection

Parameter	Optimal value	Justification
Size	45 x 45 mm	Fits safely within the Franka gripper's 50 mm stroke, providing a small margin of error to accommodate early-stage trajectory inaccuracies.
Shape	Cube	Simplifies parallel-jaw grasp, establishes strong zero-shot generalization and aids in visual alignment.
Surface	High contrast relative to surface (blue), matte	Maximizes pixel-level edge gradients, allowing the DINOv2 encoder to instantly segment the foreground object via clean attention maps.

E. Data Collection Design

As the task instruction serves as our language input, it was necessary that we defined the inputs we were going to use for our fine-tuning. OpenVLA uses the standard Llama tokenizer [14], which utilizes Byte-Pair Encoding. It segments the sentence into words or sub-words, assigning each piece a unique integer ID from its vocabulary dictionary [22]. Consequently, the structural formulation of these instructions is a critical component of VLA optimization. During the design of our data collection pipeline, we had to systematically evaluate the trade-offs of our instruction strategy: specifically, whether to introduce linguistic variation across multiple distinct tasks, or to restrict the dataset to a single, highly controlled task to first establish a reliable baseline.

Initially, our data collection encompassed recording diverse movements and generating multiple instruction variations, such as "move the cube to the right" and "lift the blue object into the red box", to test the model's baseline adaptability and prevent overfitting. However, as early training iterations yielded highly inconsistent kinematic responses, which is detailed in Section V, it became necessary to isolate the underlying pipeline variables. Consequently, we pivoted to a highly simplified approach to debug the system.

To validate our data pipeline and fine-tuning parameters, we restricted the training data to a single, fixed instruction for one task. Although mapping a trajectory to a rigid string deliberately induces overfitting, it isolates variables and ensures the core pipeline functions correctly. Our main objective at this stage was to establish a reliable baseline before introducing more complexity and generalization in the future. To support this eventual expansion, while our fine-tuning efforts focused mainly on this isolated task, we saved our original dataset comprising diverse, generalized movement episodes to facilitate future multi-task training.

For our specific task, both the cube and the box were initialized in fixed positions. We strictly mapped the trajectories to the following fixed instruction: "Place the blue cube in the red box."



Fig. 6: Experimental setup for data collection. Left: The operator's perspective during manual teleoperation. Right: The corresponding field of view from the mounted camera.

IV. IMPLEMENTATION

A. Data Processing Pipeline

The data collection pipeline consists of recording the relevant ROS topics in a rosbag, trimming the recording to the common time interval of all required topics, and converting the trimmed rosbag into an RLDS-style TensorFlow dataset for OpenVLA-OFT training. For more detail on these Python scripts, please visit our repository [23].

Approximately 52 raw demonstrations were performed for the task. The camera's view of the training is shown in Figure 6.

Rosbag and Trimming

The following ROS topics were collected using rosbag

- `/vla_front_camera/raw_image`
- `/joint_states`
- `/franka_robot_state_broadcaster/...`
  `...robot_state`
- `/servo_node/delta_twist_cmds`
- `/vla/language_instruction`
- `/tf`
- `/tf_static .`

Rosbag records the messages published on these topics together with their timestamps. It is important to note that each topic publishes at a different frequency. We also observed short delays in the recording of some topics. To avoid complications during data conversion, a trimming process was implemented to isolate the exact time interval in which all required VLA topics overlap. The recorded bag is trimmed by determining the latest first timestamp and the earliest last timestamp among the required topics and copying only the messages whose timestamps fall within this shared interval.

Rosbag to RLDS

The trimmed rosbag is then converted into an RLDS-style TensorFlow Datasets dataset. Since the VLA takes an image and a language instruction as input, the camera stream is naturally used as the temporal reference during conversion, with one RLDS step generated for each usable camera-frame interval. Because the ROS topics publish at different rates, the converter assigns the nearest subsequent message from each relevant topic to the corresponding image timestamp.

This synchronization process is illustrated in Figure 7, which compares the generated RLDS steps with the messages recorded in the original rosbag.

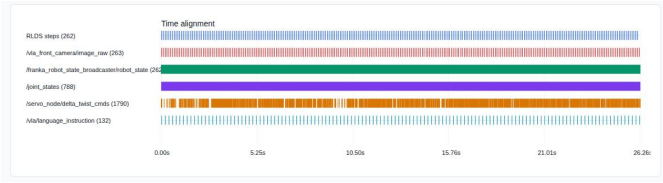


Fig. 7: Report example from a recorded episode

The images were recorded at $10\,\mathrm{Hz}$ with a resolution of $1024 \times 768\,\mathrm{px}$. During conversion, they were resized to $224 \times 224\,\mathrm{px}$, while the sampling frequency was retained at $10\,\mathrm{Hz}$, consistent with OpenVLA's recommended operating range of $5\,\mathrm{Hz}$ to $10\,\mathrm{Hz}$ [11].

The resulting step format contains the common fields of the generic RLDS step, two additional fields for the language instruction and the timestamp of the step, as well as the observations seen in 12b in the Appendix. The state vector, also called proprioception, is an 8D vector containing the end-effector XYZ position and three orientation values, roll, pitch, and yaw, followed by one padding element and the gripper width. The action is a 7D vector. Its first six elements are computed from the mean twist command between the current camera frame and the next frame, multiplied by the time difference between those frames. The final element is the gripper action, which by default is an absolute binary value representing the closed or open state.

B. Model Fine-Tuning

OpenVLA-OFT provides fine-tuning scripts in its GitHub repository [15]. To enable fine-tuning on our Franka dataset, we made the necessary modifications to these scripts. Further details are available in our repository [23].

LoRA Fine-Tuning

OpenVLA-OFT uses Low-Rank Adaptation (LoRA) [24] to efficiently adapt the pretrained 7B-parameter OpenVLA model to downstream robot datasets [15]. In conventional full fine-tuning, the pretrained weight matrices throughout the model are updated directly. Instead, for a pretrained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA keeps W_0 frozen and represents its task-specific update as a low-rank matrix $\Delta W = BA$, where $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, $r \ll \min(d,k)$. The adapted weight matrix is therefore defined as

$$W = W_0 + \Delta W = W_0 + BA.$$

During fine-tuning, only the LoRA matrices A and B are updated, reducing the number of trainable parameters for each adapted weight matrix from dk to $r(d+k)$. However, the LoRA parameters are not the only trainable parameters in OpenVLA-OFT, as newly introduced components, such as the continuous-action head and the proprioceptive-state projector, are also trained.

Dataset Transform

The OpenVLA-OFT fine-tuning pipeline expects the dataset to follow a specific structure. To support custom datasets, the authors provide a transformation script that converts the data during loading. OpenVLA-OFT expects episodes stored on disk in the TFDS/RLDS format, containing images, language instructions, proprioceptive states, and 7D actions.

During fine-tuning, the RLDS trajectory fields are standardized according to the structure shown in Figure 13. Each trajectory step is then converted into PyTorch-compatible training inputs and passed to the model. Thus, although the data are stored using a TensorFlow-based format, OpenVLA wraps them in a PyTorch dataset loader, allowing the model to be trained using PyTorch.

Fine-tuning parameters

Finding suitable fine-tuning hyperparameters played a key role in achieving successful training. Multiple hyperparameter configurations were evaluated, and the best-performing configuration found in this work is summarized in Table V.

In OpenVLA-OFT, the training duration is specified in terms of training steps. A training step should not be confused with an episode step, as introduced in subsection II-E. A training step corresponds to one optimizer update: the model processes a batch of samples, predicts the corresponding actions, computes the loss, performs backpropagation, and updates the trainable parameters.

Within the OpenVLA-OFT framework, continuous actions can be learned using either L1 regression or diffusion modeling. The authors recommend L1 regression because it achieves performance comparable to diffusion modeling while offering faster training convergence and substantially faster inference. [15]

Assuming one training process per GPU, the effective batch size is defined as

$$B_{\text{eff}} = B_{\text{GPU}} \times N_{\text{GPU}} \times N_{\text{acc}}\,,$$

where B_{GPU} is the batch size per GPU, N_{GPU} is the number of GPUs, and N_{acc} is the number of gradient accumulation steps.

For the parameters defined in Table V, the effective batch size is 24. The model therefore processes 24 samples, corresponding to 24 individual episode steps, before performing one optimizer update. This procedure is referred to as mini-batch training. The approximate number of optimizer steps required for one epoch is

$$\text{Training steps per epoch} = \frac{\text{Total training steps}}{B_{\text{eff}}}\,.$$

The maximum number of training steps can therefore be estimated as

$$\text{Max. training steps} = N_{\text{epochs}} \times \text{Training steps per epoch}\,.$$

However, the RLDS data loader shuffles and may interleave samples from the dataset. Consequently, an epoch may not always correspond to a strict pass in which every sample is seen exactly once, particularly when datasets are sampled or mixed according to predefined weights.

The learning rate and LoRA rank were adopted from the recommendations provided in [15]. The learning rate is reduced by a factor of 10 after a specified number of training steps. The decay point should be selected according to the size of the training dataset and the total number of planned optimization steps. The authors recommend continuing training until the normalized L1 training loss falls below approximately 0.01 and begins to plateau. The LoRA dropout randomly suppresses part of the LoRA adapter input during training and can therefore reduce overfitting.

FiLM, or Feature-wise Linear Modulation, is a conditioning mechanism that uses the language instruction to modulate the visual features, thereby improving the model's language-grounding ability. The augmented model that includes FiLM is denoted as OpenVLA-OFT+. In the main experiments, FiLM was used for the ALOHA setup, where multiple camera views and visually similar target objects made reliable language following more challenging.

C. Model Integration

At a high level, the system separates model inference from robot execution. The OpenVLA-OFT model is hosted in a separate GPU-backed process because loading and running the VLA model requires a dedicated machine-learning environment and substantial GPU memory. The ROS 2 bridge remains lightweight and runs on the robot-control computer, where it interfaces with the camera topics, robot-state feedback, MoveIt Servo, and the Franka gripper action server. Further details on the bridge implementation are available in our repository [23]. This separation allows the model server and the robot-control stack to run in their respective software environments while communicating through a simple HTTP interface. An overview of the complete OpenVLA-OFT inference and robot-control pipeline is provided in Figure 14 in the Appendix.

Inference

The OpenVLA-OFT server is started using the deployment script provided by [15]. Once the server is running, the ROS 2 bridge node is started. The bridge collects the latest relevant observations, including the camera image, robot state, joint states, and task instruction. The inference query rate can be configured by the user, with a default rate of $1\,\mathrm{Hz}$. Before each query, the bridge verifies that all required observations are available and sufficiently recent. The observations are then converted into the payload format expected by the inference server.

If no action sequence is currently being executed, the bridge sends an HTTP request to the `POST /act` endpoint. In our setup, the measured inference latency was approximately $0.155\,\mathrm{s}$. Inference was performed without model quantization.

ROS 2 Control & Execution

After predicting an action chunk of length eight, the OpenVLA-OFT server returns it to the ROS 2 bridge node. The bridge selects a configurable number of actions from the returned chunk, stores them in a queue, and executes them sequentially. While an action sequence is active, further inference requests are blocked. If a different task instruction is received during execution, the remaining queued actions are discarded and the current sequence is canceled.

In the default `integrated_delta` mode, the first six action dimensions are interpreted as desired end-effector pose increments: three translational increments and three rotational increments. Since MoveIt Servo expects velocity commands in the form of a `geometry_msgs/TwistStamped` message, the bridge converts these increments into linear and angular velocities by dividing them by the command duration Δt. Both the command duration and the number of actions executed from each chunk can be configured through ROS 2 parameters, which may be supplied as command-line arguments. The default is $\Delta t = 0.2\,\mathrm{s}$ and 8 actions.

The resulting linear and angular velocities are published to the Servo command topic, while the final action dimension is handled separately as the gripper command. For each selected action, the bridge repeatedly publishes the corresponding `TwistStamped` message at approximately $29.4\,\mathrm{Hz}$ for the specified command duration Δt. Once this command interval has elapsed, the next action in the queue is executed. After the complete action sequence has finished, the bridge publishes a zero-twist command unless the current command has already been replaced by a newer action.

V. RESULTS & DISCUSSION

This section delves into the empirical results obtained across four primary fine-tuning iterations of the OpenVLA model, alongside the iterative pipeline adjustments that needed to be implemented in response to observed failure modes. A summary of these training configurations is provided in Table II.

Before training the model on a broader, multi-task dataset, we first investigated whether OpenVLA could be fine-tuned successfully for a single, specific manipulation task. Therefore, the first iteration, referred to as Training 1, focused on picking up the cube and moving it laterally. The model was trained on 106 episodes for 50 epochs. During inference, the robotic arm demonstrated rudimentary object-tracking behavior by consistently orienting its end-effector toward the target cube. However, it made only limited progress toward completing the physical manipulation task. The manipulator decelerated considerably as it approached the cube and eventually stopped before grasping it.

Although the correct directional movement provided an initial indication that the model was learning from the custom dataset, its inability to complete the task demonstrated that further dataset refinement was required. To determine whether the model had learned language-conditioned behavior rather than simply memorizing the demonstrated trajectories, it was also evaluated using basic commands such as "close the gripper". Regardless of the instruction, however, the robot continued to orient its gripper toward the cube. This suggested that the model had overfitted to the limited trajectory distribution and had learned to reproduce the dominant motion pattern instead of grounding its actions in the language instruction.

For Training 2, we therefore assembled a more general dataset composed of the initially recorded trajectories together with demonstrations for basic commands, including "close the gripper", "move up", and "pick up the cube". However, the resulting action generation was highly inconsistent. When provided with a task instruction, the manipulator would occasionally move toward the target cube, generate uncoordinated movements in seemingly arbitrary directions, similar to the behavior of the non-fine-tuned baseline model, or fail to actuate entirely.

TABLE II: Overview of Model Training Iterations

Training	Dataset Description	Number of Episodes	Number of Epochs
1	Picking up the cube and moving it to the right/left	106	50
2	All recorded data (excluding dataset using the red box)	135	50
3	Putting the cube in the box (with near-zero actions)	52	20
4	Putting the cube in the box (with near-zero actions removed)	52	60



Fig. 8: The image shows the stopping position of the robot after being given the instruction to place the cube in the box.

For Training 3, we returned to the controlled single-task approach used in Training 1. The objective was to deliberately encourage overfitting to a specific task and instruction in order to verify the functionality of the data-processing and fine-tuning pipelines. The original task of moving the cube laterally was replaced with the more specific task of placing the cube inside the box. All 52 episodes were assigned the same fixed instruction: "Place the blue cube in the red box".

This red box provided a more distinctive visual cue. In addition, both objects were initialized at fixed positions on the table. After 20 training epochs, the resulting robot trajectories were noticeably smoother and more consistent. The gripper successfully navigated toward the target and stopped only a few millimeters away from the cube, as shown in Figure 8. Despite the improved spatial accuracy, the manipulator repeatedly hovered above the cube and abruptly stopped without closing the gripper.

This behavior was hypothesized to originate from the human teleoperation demonstrations. During data collection, the operators systematically decelerated the robot near the cube to align the gripper carefully before grasping it. Consequently, the dataset contained a large number of near-zero action samples around this position. The model may therefore have learned that approaching the cube should be followed by little or no movement rather than by the subsequent grasping action. Furthermore, the training L1 loss plateaued at approximately 0.0369, remaining above the recommended value of 0.01 for well-converged fine-tuning.

To mitigate the "freezing" phenomenon identified in the third iteration, a data-processing script was written to algorithmically filter out near-zero action samples during the transformation from rosbags to the RLDS dataset format. Utilizing this newly filtered dataset, the fourth training was conducted. To ensure the model internalized the complex, multi-step sequence of actions required for successful prehension and to allow the L1 loss to successfully converge below the 0.01 plateau, the training duration was extended to 60 epochs. Table III summarizes the results of 10 trials with the final model.

The formulation of the task was not a deciding factor in the outcome as both "pick up the cube" and "place the blue cube in the box" yielded successful results. We hypothesize that while the model was deliberately overfit to the phrase "Place the blue cube in the red box", it successfully executed the task when prompted with synonymous variations. We think this might be due to the fact that the underlying foundation model retained its pre-trained semantic understanding. The base model might already know that "pick up" and "place in" are semantically linked to the actions we trained it on, allowing for an "emergent zero-shot generalization" despite our rigid fine-tuning.

TABLE III: Evaluation of 10 Physical Validation Trials

Outcome	Count	Observations & Failure Modes
Success	7	Task executed successfully after the manual hover override.
Failure	3	• Dropped cube mid-trajectory, but continued to the box and opened the gripper. • Failed to grip the cube initially (required manual grasp override), then failed to open the gripper inside the box. • Dropped the cube and failed to hover above the target box.

VI. CONCLUSION & FUTURE WORK

This final chapter draws a close to the evaluation of our task, and is divided into two main sections. First, the conclusion summarizes the key engineering findings and assesses the practical viability and adaptation of our system. Following this, the future works section outlines further potential research, proposing enhancements in data augmentation, image acquisition, and acoustic interaction to further refine the system's robustness and scalability.

A. Conclusion

This project developed and integrated a language-conditioned manipulation pipeline for the Franka Emika

Panda robotic arm operating within a ROS 2 environment. By addressing the practical engineering challenges of deployment on our specific hardware, we established a synchronized data collection architecture capable of translating visual observations and text instructions directly into continuous robotic actions. Through this framework, our implementation fine-tuned the OpenVLA-OFT architecture on a custom dataset.

Evaluation of the fine-tuned model demonstrated that the OpenVLA architecture can successfully learn from localized datasets, as the robotic system developed rudimentary object tracking and decent directional inference. Initial testing highlighted critical challenges in translating human teleoperation into continuous control, where human-introduced deceleration during data collection caused the model to replicate pauses and freeze directly above the target object. However, by algorithmically filtering these near-zero action samples and extending training durations to allow the L1 loss to converge, the model successfully overcame these teleoperation biases. In final validations, the robotic arm executed the full physical manipulation task, requiring only a minor manual mechanical intervention to bypass residual hesitation and trigger the subsequent trajectory phase, specifically, a slight manual push when the arm hovered above the object and the target box.

This work validates the fundamental viability of deploying generalized VLA models onto physical control systems. The established data processing and integration pipeline successfully bridged the gap between large-scale models and practical hardware. Future iterations of this system can focus on further refining the demonstration dataset and training parameters to completely eliminate the residual hesitation with the goal of achieving fully autonomous, seamless task execution. While single-instruction fine-tuning temporarily simplified the interaction into a functional imitation learning stack, this deliberate constraint was essential for isolating control latency and hardware parameters before scaling up linguistic and task diversity.

B. Future Work

Generalization

While this project demonstrates the feasibility of fine-tuning a VLA model for a specific manipulation task, a critical avenue for future work is establishing robust generalization. In real-world robotic applications, environments are inherently unstructured and dynamic, meaning deployed systems will inevitably encounter novel objects, unseen spatial configurations, and varying visual conditions. Generalization is therefore not merely a preferred attribute, but a strict necessity to prevent models from becoming brittle and overly reliant on narrow, task-specific datasets [12]. Future research should explore multi-task training and leverage larger and more diverse robotic datasets to enhance the zero-shot capabilities of the architecture [16]. By prioritizing generalization, the model can fully exploit the broad semantic priors inherited from its vision-language backbone, ultimately leading to highly adaptable control systems capable of executing a wide array of novel instructions without the need for continuous retraining [9].

Kinematic and Semantic Data Augmentation

To scale the model's generalization capabilities without the high physical overhead of manual teleoperation, future development may focus on a data mirroring pipeline. By laterally flipping the RGB visual observations from the camera and simultaneously swapping directional tokens (for example: altering "move left" to "move right") within the language instructions, the effective dataset size can be doubled synthetically. This would require a synchronized mathematical inversion of the continuous action space. To maintain physical realism, the translational axis corresponding to the camera's lateral plane must be negated, alongside the inversion of the associated rotational states, which would ensure the manipulator's end-effector trajectory remains perfectly aligned with the augmented visual and semantic reality. A framework called MirrorDuo, which is depicted in Figure 9, has explored this concept as a data augmentation strategy [25].

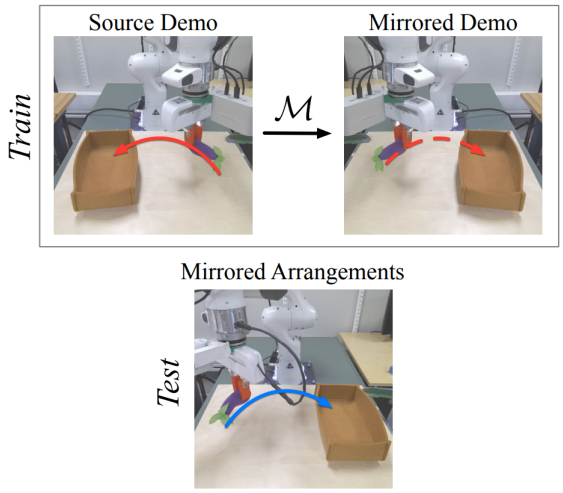


Fig. 9: MirrorDuo: training on mirrored paths and testing mirrored movements.

Multi-View and Depth Perception

The current architecture relies on a single monocular observation, which can introduce spatial obstruction and depth ambiguity during grasping tasks. To address this, future improvements can explore the integration of secondary visual inputs. The addition of an eye-in-hand camera mounted directly on the gripper would provide localized context during the critical approach and interaction phases, such as when picking up the block [26]. Furthermore, incorporating external or depth sensors could facilitate a transition from 2D pixel-space feature extraction to true 3D scene understanding, heavily improving the system's spatial reasoning and collision avoidance.

Voice-Conditioned Control via Speech-to-Text

Another potential improvement to the system would be an upgrade from the static text-based instruction interface to an active voice-command system, resulting in a seamless and hands-free human-robot interaction. By deploying a low-latency Automatic Speech Recognition (ASR) model as a

dedicated and concurrent ROS 2 node, human speech can be dynamically transcribed into textual prompts [27]. This integration would allow users to naturally dictate tasks, interrupt execution, or provide real-time corrections to the VLA model without requiring a physical keyboard interface, significantly streamlining the operational workflow.

Researchers created VLAS [27], which is a model that processes raw audio and images at the same time, allowing the robot to comprehend natural voice commands and translate them into physical movements. An impressive feature of the model is its ability to differentiate voices, allowing it to handle vaguer instructions like "pick up my cup". Since the paper explicitly states "It is important to highlight our method is orthogonal to other VLA models, and thus, can be combined with them to achieve superior performance", this model could be combined with our existing one to allow speech instruction as an option.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017.

[2] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[4] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, "Dinov2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=a68SUt6zFt

[5] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2019.

[6] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[8] F. Bordes, R. Y. Pang, A. Ajay, A. C. Li, A. Bardes, S. Petryk, O. Mañas, Z. Lin, A. Mahmoud, B. Jayaraman, M. Ibrahim, M. Hall, Y. Xiong, J. Lebensold, C. Ross, S. Jayakumar, C. Guo, D. Bouchacourt, H. Al-Tahan, K. Padthe, V. Sharma, H. Xu, X. E. Tan, M. Richards, and S. Lavoie, "An introduction to vision-language modeling," *arXiv*, 2024.

[9] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," *arXiv preprint arXiv:2307.15818*, 2023.

[10] M. U. Din, W. Akram, L. S. Saoud, J. Rosell, and I. Hussain, "Vision language action models in robotic manipulation: A systematic review," *arXiv preprint arXiv:2507.10672*, 2025. [Online]. Available: https://arxiv.org/abs/2507.10672

[11] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[12] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[13] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "π_0: A Vision-Language-Action Flow Model for General Robot Control," *arXiv preprint arXiv:2410.24164*, 2024.

[14] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[15] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025.

[16] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6892–6903.

[17] S. Ramos, S. Girgin, L. Hussenot, D. Vincent, H. Yakubovich, D. Toyama, A. Gergely, P. Stanczyk, R. Marinier, J. Harmsen *et al.*, "Rlds: an ecosystem to generate, share and use datasets in reinforcement learning," *arXiv preprint arXiv:2111.02767*, 2021.

[18] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074

[19] Franka Emika GmbH, *Product Manual Franka Emika Robot*, 2021, version 10.21. [Online]. Available: https://download.franka.de/documents/100010_Product%20Manual%20Franka%20Emika%20Robot_10.21_DE.pdf

[20] SVPRO, "5MP 30FPS USB Camera, Wide Angle High Resolution Security Camera for Laser Engraver and Industry (ASIN: B0D8SQV28P)," https://www.amazon.de/Weitwinkel-Hochaufl%C3%B6sende-Sicherheitskamera-Lasergravierer-Industrie/dp/B0D8SQV28P/, 2026, [Online; accessed 15-July-2026].

[21] D. Niu, Y. Sharma, B. Shi, R. Ding, M. Gioia, H. Xue, H. Tsai, K. Kallidromitis, A. Pai, C. Regan *et al.*, "Learning to grasp anything by playing with random toys," *arXiv preprint arXiv:2510.12866*, 2025.

[22] D. Shin, "Enhancing linguistic generalization of vla: Fine-tuning openvla via synthetic instruction augmentation," 2026.

[23] G. V. Chandran, K. Gamaleldin, L. Zeghib, and R. Mahale, "Franka-OpenVLA: Repository," 2026. [Online]. Available: https://github.com/KhadijaGamaleldin/franka-openvla

[24] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models." *Iclr*, vol. 1, no. 2, p. 3, 2022.

[25] Z. Zhuang, R. Wang, G. L. Marchetti, F. T. Pokorny, and D. Kragic, "MirrorDuo: Reflection-Consistent Visuomotor Learning from Mirrored Demonstration Pairs," in *Conference on Robot Learning*, 2025. [Online]. Available: https://openreview.net/forum?id=s6W4a1yK4x

[26] M. J. Kim, J. Wu, and C. Finn, "Giving robots a hand: Learning generalizable manipulation with eye-in-hand human video demonstrations," *arXiv preprint arXiv:2307.05959*, 2023.

[27] W. Zhao, P. Ding, M. Zhang, Z. Gong, S. Bai, H. Zhao, and D. Wang, "VLAS: Vision-Language-Action Model with Speech Instructions for Customized Robot Manipulation," in *International Conference on Learning Representations*, 2025.

APPENDIX

Selection criterion	**OpenVLA** [11]	**Octo** [12]	π_0/**OpenPI** [13]
Documentation and online resources	Extensive documentation, Hugging Face integration, inference examples, LoRA and full fine-tuning instructions, custom-dataset support, and troubleshooting resources. Primarily implemented in PyTorch.	Good documentation, pretrained checkpoints, and examples for inference, fine-tuning. The main implementation is based on JAX.	Good documentation. OpenPI includes JAX and PyTorch implementations, Docker support, LeRobot dataset integration, remote inference, and examples for LIBERO, ALOHA, DROID, and UR5.
Diversity of training data	Trained on approximately 970,000 robot episodes from OXE, covering multiple robot embodiments, tasks, scenes, and objects.	Trained on approximately 800,000 trajectories from OXE, covering multiple robots, tasks, scenes, observation configurations, and action spaces.	Pretrained on more than 10,000 hours of robot data. The reported mixture includes data from seven robot configurations and 68 internally collected tasks, together with OXE data from additional robot embodiments.
Model size	7B parameters.	Octo-Small has 27M parameters and Octo-Base has 93M parameters.	3.3B parameters.
Inference GPU memory	Moderate to high. 7 GB (with quantization, INT4) to 15 GB (without quantization, BF16). OpenVLA reported no performance trade-off when using memory-efficient inference via quantization.	Low. No official minimum is reported. Due to its small model size, Octo has the lowest inference requirements.	Moderate. The official repository estimates > 8 GB for inference.
Fine-tuning options and GPU memory	LoRA fine-tuning requires approximately > 27 GB of GPU memory, while full fine-tuning generally requires a multi-GPU server.	Supports head-only, head-MLP-only, and full-model fine-tuning.	The official repository estimates > 22.5 GB for LoRA fine-tuning and > 70 GB for full fine-tuning. (LoRA only supported by the JAX implementation)

TABLE IV: Comparison of publicly available VLA models.

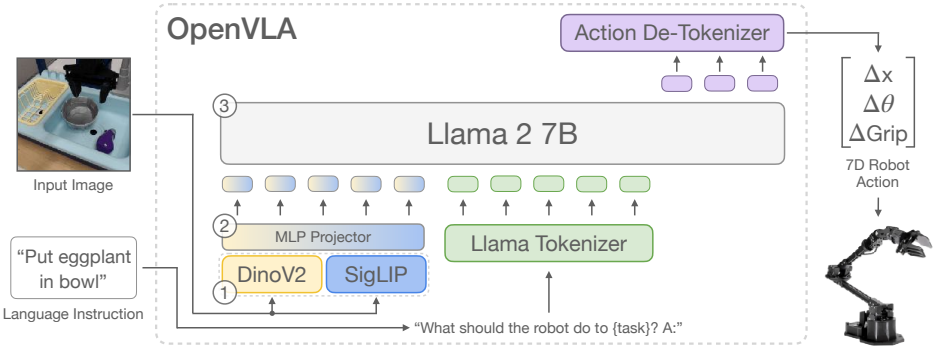


Fig. 10: OpenVLA architecture [11]

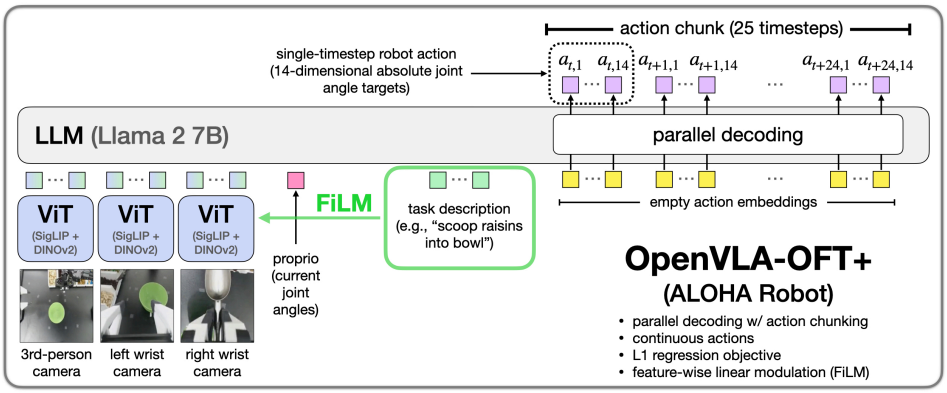
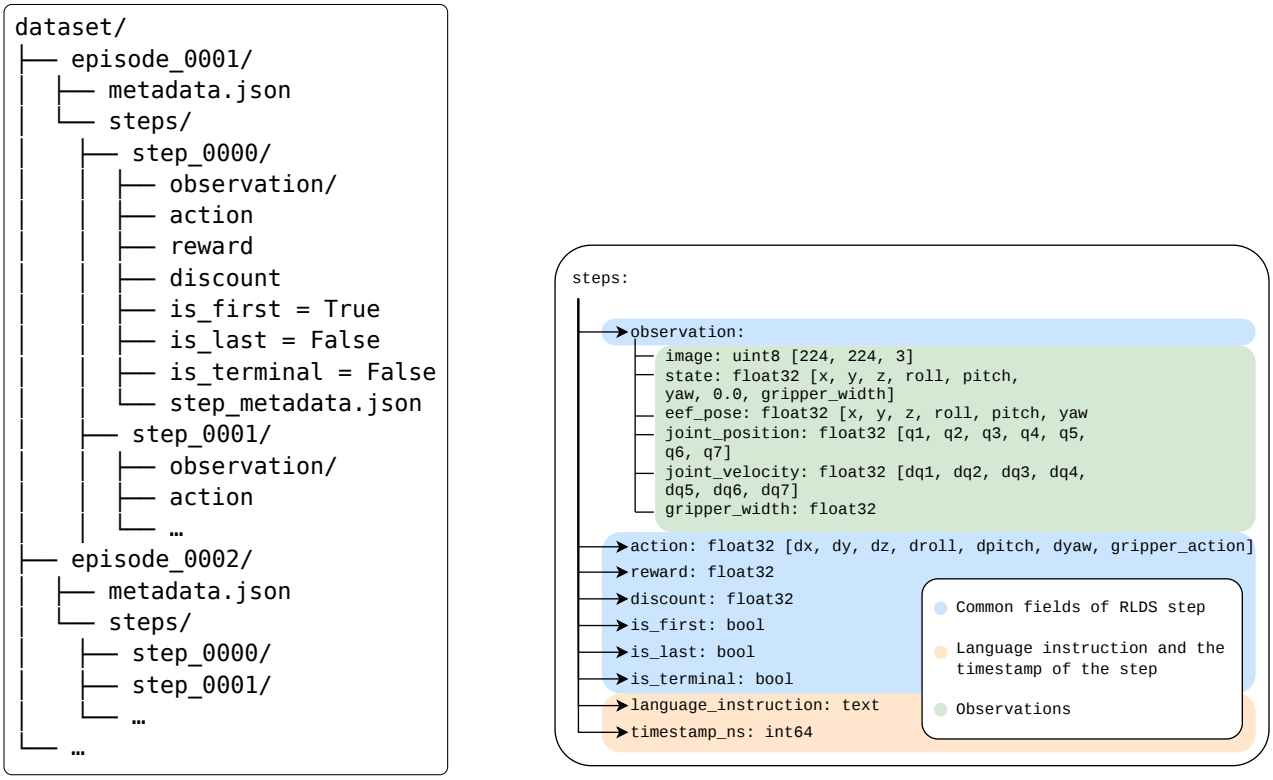


Fig. 11: OpenVLA-OFT+ architecture [15]



(a) RLDS generic format

(b) Our step format

Fig. 12: RLDS format

```
{
  "observation": {
    "image_primary": ...,
    "proprio": ...,
    "timestep": ...
  },
  "task": {
    "language_instruction": ...
  },
  "action": ...,
  "dataset_name": ...
}
```

Fig. 13: OpenVLA RLDS format

TABLE V: **OpenVLA-OFT+ fine-tuning hyperparameters for the task "place the blue cube in the red box." with near-zero actions removed.** The model is fine-tuned using continuous-action prediction with an L1 regression objective and LoRA adaptation.

Hyperparameter	**Value**
Base model	`openvla/openvla-7b`
Number of GPUs	3 x NVIDIA A100 (80 GB)
Learning objective	L1 regression
Use diffusion	False
Use proprio	True
Use FiLM	False
Number of input images	1
Batch size	8
Gradient accumulation steps	1
Effective batch size B_{eff}	24
Learning rate	5×10^{-4}
Steps before learning-rate decay	20000
Maximum training steps	28850
Number of epochs N_{epochs}	60
Shuffle-buffer size	13150
Image augmentation	True
LoRA rank	32
LoRA dropout	0.0

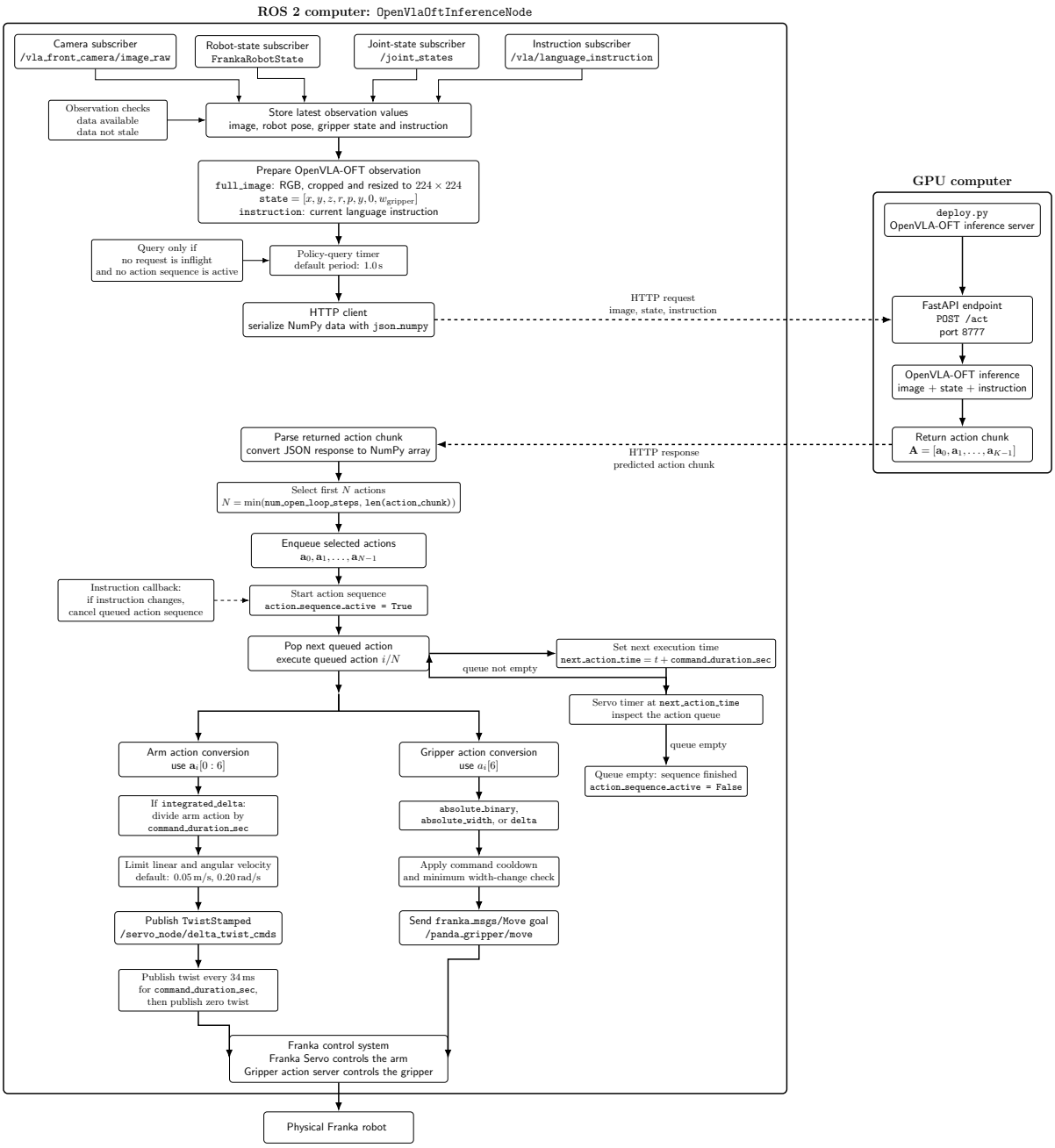


Fig. 14: OpenVLA-OFT inference and robot-control pipeline